

Tooling From Embeddings

if we run locally llm model of 1.5B, can we use embeddings to add tooling and other missing capabilities to it? If yes, can we create embeddings for adding for same model but variant of 70B or more strong? If this is all possible we need step by step guide on how to do this using llama.cpp and other proper tools we may need.

Yes, you can use embeddings to add tooling and other capabilities to a 1.5B model. This is typically done through a technique called Retrieval-Augmented Generation (RAG), where you use an embedding model to create a knowledge base (a vector store), and then you can also enable the model to call external functions (known as "function calling" or "tool use").

However, you cannot directly reuse embeddings generated for one model with another (like a 70B variant). They are incompatible because each model has its own unique embedding space.

Below is a step-by-step guide to achieving this locally using llama.cpp and other tools.

Step-by-Step Guide: Building a Tool-Enhanced Local AI

Step 1: Setting Up Your Environment

1. Install llama.cpp and llama-cpp-python: · First, clone the llama.cpp repository and build it:

```
git clone https://github.com/ggerganov/llama.cpp
cd llama.cpp
make -j # Or use CMake for a more robust build
```

- Then, install the Python bindings:

```
pip install llama-cpp-python
```

2. Install Supporting Libraries: We'll use langchain for the RAG pipeline and chromadb as our vector store.

```
pip install langchain langchain-community sentence-transformers chromadb
pypdf
```

Step 2: Choosing and Downloading Models

1. Get Your 1.5B LLM (GGUF Format): · You need a 1.5B parameter model in GGUF format for your main chat/instruction model. You can find many on Hugging Face, such as from Qwen2.5 or Phi-3 families. For this example, we'll use a 7B model just to illustrate, but the process is identical for a 1.5B model. · Option 1 (Python): Download it programmatically:

```
import os
import requests
from tqdm import tqdm

model_path = "qwen2.5-1.5b-instruct-q4_k_m.gguf" # Example, replace with
actual URL
if not os.path.exists(model_path):
    print(f"Downloading {model_path}...")
    model_url = "https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-
GGUF/resolve/main/qwen2.5-1.5b-instruct-q4_k_m.gguf"
    response = requests.get(model_url, stream=True)
    total_size = int(response.headers.get('content-length', 0))
    with open(model_path, 'wb') as f:
        for data in tqdm(response.iter_content(chunk_size=1024),
total=total_size//1024):
            f.write(data)
```

- Option 2 (CLI): Use llama-cli to download directly from Hugging Face:

```
./llama-cli -hf Qwen/Qwen2.5-1.5B-Instruct-GGUF:q4_k_m
```

2. Get a Dedicated Embedding Model: For best results, use a model specifically trained for embeddings. A great option is nomic-embed-text-v1.5.

```
# Download the GGUF file for the embedding model
wget https://huggingface.co/nomic-ai/nomic-embed-text-v1.5-
GGUF/resolve/main/nomic-embed-text-v1.5.Q4_K_M.gguf
```

Step 3: Building the Knowledge Base (RAG)

1. Load and Process Documents:

```
from langchain.document_loaders import TextLoader, PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Load your documents (example with a text file)
loader = TextLoader("path/to/your/document.txt")
documents = loader.load()

# Split documents into manageable chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50)
texts = text_splitter.split_documents(documents)
```

2. Create Embeddings and Store Them:

```

from langchain.embeddings import LlamaCppEmbeddings
from langchain.vectorstores import Chroma

# Initialize the embedding model
embeddings = LlamaCppEmbeddings(model_path="nomic-embed-text-
v1.5.Q4_K_M.gguf")

# Create a vector store and index the documents
vectorstore = Chroma.from_documents(texts, embeddings)

```

Step 4: Implementing Tool Use (Function Calling)

This is where you teach your LLM to use external tools. You'll define the tools and use llama.cpp's built-in function calling features.

1. Define Your Tools: Create a list of tools with a name, description, and JSON schema for parameters.

```

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_weather",
            "description": "Get the current weather in a given location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "The city and state, e.g. San
Francisco, CA",
                    },
                },
                "required": ["location"],
            },
        },
    },
]

```

2. Create a Function Handler: This function will execute when the model calls a tool.

```

def handle_tool_call(tool_name, arguments):
    if tool_name == "get_current_weather":
        # In a real scenario, you'd call a weather API here
        return f"The weather in {arguments['location']} is sunny with a
high of 72°F."
    return "Unknown tool"

```

3. Integrate into a Conversation Loop: The basic flow is: User Input -> LLM decides to call a tool -> Your code executes the tool -> The result is fed back to the LLM.

```
from llama_cpp import Llama

# Initialize your 1.5B LLM
llm = Llama(model_path="qwen2.5-1.5b-instruct-q4_k_m.gguf", n_ctx=2048,
n_gpu_layers=-1)

messages = [{"role": "user", "content": "What's the weather like in
London?"}]

response = llm.create_chat_completion(messages=messages, tools=tools,
tool_choice="auto")

if response['choices'][0]['message'].get('tool_calls'):
    tool_call = response['choices'][0]['message']['tool_calls'][0]
    function_name = tool_call['function']['name']
    arguments = json.loads(tool_call['function']['arguments'])

    # Execute the function
    function_response = handle_tool_call(function_name, arguments)

    # Add the function result to the conversation
    messages.append(response['choices'][0]['message'])
    messages.append({
        "role": "tool",
        "name": function_name,
        "content": function_response
    })

    # Get the final response from the LLM
    final_response = llm.create_chat_completion(messages=messages)
    print(final_response['choices'][0]['message']['content'])
else:
    # No tool call needed, print the direct response
    print(response['choices'][0]['message']['content'])
```

Step 5: Putting It All Together in an Agent

A "ReAct" (Reasoning and Acting) agent uses both RAG and tool calling in a unified way. You can use langchain to orchestrate this.

```
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain_community.llms import LlamaCpp
from langchain.memory import ConversationBufferMemory

# Define a tool for the RAG system
def rag_search(query):
```

```

docs = vectorstore.similarity_search(query, k=2)
return "\n\n".join([doc.page_content for doc in docs])

rag_tool = Tool(
    name="Knowledge Base Search",
    func=rag_search,
    description="Useful for answering questions about your personal documents."
)

# Define a weather tool (similar to before)
weather_tool = Tool(
    name="Weather Lookup",
    func=handle_tool_call, # You would need to adapt this to a proper function
    description="Useful for getting current weather information."
)

# Initialize the LLM and the agent
llm = LlamaCpp(model_path="qwen2.5-1.5b-instruct-q4_k_m.gguf", n_ctx=2048,
n_gpu_layers=-1)
agent = create_react_agent(llm, [rag_tool, weather_tool], prompt) # You need to
define a 'prompt'
agent_executor = AgentExecutor(agent=agent, tools=[rag_tool, weather_tool],
verbose=True, memory=memory)

# Run the agent
agent_executor.invoke({"input": "Summarize the latest updates from my project
document and tell me the weather in New York."})

```

⚠ Important Considerations for 1.5B Models

- **Function Calling:** While llama.cpp supports the mechanics of function calling, a 1.5B model may not have been fine-tuned for it. It might require very explicit prompting or may not understand the tool syntax at all. For reliable tool use, consider models like functionary or larger, instruction-tuned models.
- **Performance:** A 1.5B model will be fast but its reasoning and instruction-following abilities will be limited compared to a 70B model.

🔄 Handling the 70B Variant Question

You cannot reuse the vector database created with nomic-embed-text-v1.5 if you switch your main LLM to a 70B model for chat generation. This is because the embedding model and the generative LLM are separate and perform different tasks.

However, the RAG pipeline is model-agnostic. The knowledge base built with the nomic-embed model can be used to provide context to any LLM, whether it's your 1.5B model or a 70B model. The process of retrieving relevant documents from the vector store is independent of the LLM that will later process that context.

If you have further questions about implementing any of these steps, feel free to ask.

Can it work on weaker machines?

Can this then work on the following host machine: Amd Ryzen9, 32 Giga of Ram, RTX graphics with 6 gigs of ram, 2 tb ssd nvme2 ? If yes would it be usable and have reasonable speed of execution?

Yes, absolutely. That host machine is very well suited for this task. In fact, it's overkill for the 1.5B model but provides a great foundation if you ever want to experiment with larger models.

Here is the detailed breakdown of performance expectations and how to maximize speed on your specific hardware.

✓ Hardware Compatibility & Performance Forecast

Component Role in this Setup Expected Performance AMD Ryzen 9 Handles document parsing, orchestrates tools, and runs CPU fallback layers. Idle to low usage. The GPU will handle almost 100% of the model work. RTX 6GB VRAM Runs the 1.5B LLM AND the Embedding Model simultaneously. Extremely Fast. A 1.5B Q4 model is ~1GB. It fits entirely in your 6GB VRAM. 32 GB RAM Stores the Vector Database (ChromaDB) and system memory. Snappy. Retrieval from an NVMe-backed vector store will feel instantaneous. 2 TB NVMe SSD Stores the model files and the vector index. Fast Loading. Model loading time will be ~1-2 seconds.

🦋 Speed Expectations (Tokens Per Second - TPS)

Since your RTX card has 6GB VRAM and the 1.5B model is tiny, you can offload 100% of the model layers to the GPU.

- Pure GPU Inference (llama.cpp with CUDA): · 1.5B Model (Q4_K_M): 150 - 300+ tokens per second. · User Experience: This is faster than you can read. Responses to "What's the weather?" will appear instantly after the tool call executes.
- Embedding Generation (nomic-embed-text-v1.5): · Speed: ~10-20 documents per second. · User Experience: Indexing 1,000 pages of text will take about 1-2 minutes.
- RAG Retrieval (ChromaDB): · Latency: < 50 milliseconds to find relevant chunks.

⚙️ Critical Configuration: Enabling CUDA for Maximum Speed

By default, llama-cpp-python might install a CPU-only version. You must install the CUDA-enabled version to use your RTX card.

Option 1: Reinstall with CUDA Flags (Recommended) Run this in your terminal:

```
# Uninstall the CPU version first
pip uninstall llama-cpp-python -y

# Set environment variable for CUDA and reinstall
CMAKE_ARGS="-DGGML_CUDA=on" pip install llama-cpp-python --no-cache-dir --force-reinstall --verbose
```

Option 2: Verify GPU is Detected in Code

```
from llama_cpp import Llama

# Load model, force all layers to GPU (-1 means "all")
```

```
llm = Llama(
    model_path="qwen2.5-1.5b-instruct-q4_k_m.gguf",
    n_gpu_layers=-1, # <--- CRITICAL: Offload everything to RTX
    n_ctx=4096       # Context window size
)

# You should see output in terminal like: "ggml_cuda_init: found 1 CUDA
devices"
```

🔗 What About the 70B Variant? (The 6GB VRAM Reality Check)

You asked if the embeddings can be reused for a 70B model. Yes, the knowledge base stays the same. But you asked if it works on this machine. The answer is Yes, but slower.

· LLM Inference for 70B: The 70B model file (Q4) is ~40GB. It will not fit in your 6GB VRAM. · How it runs: You would set `n_gpu_layers=20` (or similar) to put only a few layers on the RTX. The rest runs on the Ryzen CPU using 32GB RAM. · Performance: Expect 1 to 3 tokens per second. It's usable for a slow, thoughtful assistant, but it will feel like typing speed, not instant chat.

🔗 Step-by-Step Setup Summary for Your Machine

1. Install CUDA Toolkit (if not already): `sudo apt install nvidia-cuda-toolkit` (Linux) or ensure NVIDIA drivers are up to date (Windows).
2. Build llama.cpp with CUDA (using the `CMAKE_ARGS` command above).
3. Download the 1.5B Model (e.g., Qwen2.5-1.5B-Instruct-Q4_K_M.gguf).
4. Set `n_gpu_layers=-1` in your Python script.

Verdict: This is an ideal, high-speed setup for a local 1.5B model with RAG and tool use. The 6GB VRAM is the sweet spot for this size class. Enjoy the lightning-fast responses

Model suggestions

Regarding the "⚠ Important Considerations for 1.5B Models · Function Calling: While llama.cpp supports the mechanics of function calling, a 1.5B model may not have been fine-tuned for it. It might require very explicit prompting or may not understand the tool syntax at all. For reliable tool use, consider models like functionary or larger, instruction-tuned models. · Performance: A 1.5B model will be fast but its reasoning and instruction-following abilities will be limited compared to a 70B model." <--- Please give us the list of suggested models - exact full names as we would get them from the huggingface!

Here is a list of models with native function-calling capabilities, compatible with `llama.cpp`. All are available in the **GGUF format** on Hugging Face. I've focused on models that offer a good balance between performance and the ability to run on your local machine (6GB VRAM, 32GB RAM).

🔗 Suggested Models for Native Function Calling (GGUF Format)

Model (Hugging Face ID)	Size	VRAM	Notes
		Use (Q4_K_M)	

Model (Hugging Face ID)	Size	VRAM Use (Q4_K_M)	Notes
itlwas/Arch-Function-1.5B-Q4_K_M-GGUF	1.5B	~1 GB	Fast, fine-tuned from Qwen2.5-1.5B for function calling. Excellent for 6GB VRAM.
LiquidAI/LFM2-1.2B-Tool-GGUF	1.2B	< 1 GB	Designed for edge devices, fast, non-reasoning model for concise tool use.
Mungert/LFM2-1.2B-Tool-GGUF	1.2B	< 1 GB	Community fine-tune of LFM2-1.2B for tool calling.
unclecode/tinyllama-function-call-Q4_K_M-GGUF-250424	1.1B	~0.7 GB	TinyLlama fine-tuned for function calling, very fast.
smcleod/functionary-small-v3.2-Q6_K-GGUF	~8B	~5 GB	Well-regarded "Functionary" model, specifically designed for robust function calling.
unclecode/llama3-function-call-Q4_K_M-GGUF-240424	8B	~5 GB	Llama-3-8b fine-tuned for function calling, also supports TinyLlama variant.
bullerwins/Qwen2.5-7B-Instruct-GGUF	7B	~4.5 GB	Qwen2.5-7B, strong general instruction model with native function-calling.
asiansoul/meta-llama-3.1-8b-instruct-GGUF	8B	~5 GB	Meta's official Llama-3.1, a strong general-purpose model with function-calling.

How to Use Them

You can use any of these models directly with `llama.cpp`:

```
# Download and run (example with the 1.5B Arch Function model)
llama-cli -hf itlwas/Arch-Function-1.5B-Q4_K_M-GGUF -p "Your prompt here"
```

For Python, you can use `llama-cpp-python` as before, just replace the model path.

What to Expect from Different Model Sizes

Given your hardware, the models above fall into two categories:

- **Small Models (~1-2B):** These will fit completely in your 6GB VRAM (`n_gpu_layers=-1`), offering extremely fast responses (likely **>100 tokens per second**). Their function-calling ability is purpose-built but their general world knowledge and conversational nuance will be more limited.
- **Medium Models (~7-8B):** These will require a mix of GPU and CPU memory (`n_gpu_layers` around 20-30). They will be significantly smarter and more capable conversationalists, but slower

(perhaps **5-15 tokens per second** on your CPU/GPU combo). The **functionary** and **Llama-3.1** models in this category are known for very reliable tool use.

I recommend starting with one of the smaller, purpose-built models (like the 1.5B **Arch-Function**) for the fastest experience. You can always download one of the 7-8B models later if you find you need a smarter brain for more complex tasks.

Let me know if you'd like more specific instructions on running any of these!

Will it work with suggested models?

Using these suggested models would solve the problem and tool cooling in combination with embeddings would 100% work and model would be able to write and read files and other programming tasks?

Yes, this combination is perfectly suited to that task. The models I suggested can act as the "brain" that decides to use a tool, while the actual file reading/writing is handled by Python functions you create.

It's crucial to understand the division of labor: the local LLM is an API caller, not an executor. It will analyze your prompt, decide if a tool is needed, and generate the command (e.g., `write_file("notes.txt", "Hello world")`). Your Python script then **executes** this command, returning the result to the model to maintain the conversation.

The models **Arch-Function-1.5B** and **LFM2-1.2B-Tool** are specifically fine-tuned to reliably output these "function calls". The **LFM2** model, for instance, can output calls like `[get_candidate_status(candidate_id="12345")]`.

Building a File-System Aware Agent

You'll need to create a **Python script** that acts as a bridge between the model and your computer. Here's a conceptual example of how to define tools and handle the model's requests:

1. Defining Your Tools The model needs to know what actions it can perform. You provide this by defining a **tools** list with a clear **name**, **description**, and **parameters** for each function.

```
# Example tool definitions
tools = [
    {
        "type": "function",
        "function": {
            "name": "read_file",
            "description": "Read the contents of a file from the local filesystem.",
            "parameters": {
                "type": "object",
                "properties": {
                    "file_path": {"type": "string", "description": "The path to the file to read."}
                },
                "required": ["file_path"]
            }
        }
    }
]
```

```

    },
    {
        "type": "function",
        "function": {
            "name": "write_file",
            "description": "Create a new file or completely overwrite an
existing one with specified content.",
            "parameters": {
                "type": "object",
                "properties": {
                    "file_path": {"type": "string", "description": "The path to
the file to write."},
                    "content": {"type": "string", "description": "The content
to write to the file."}
                },
                "required": ["file_path", "content"]
            }
        }
    },
    # You can also add more complex tools like `list_files` or `execute_python`
]

```

2. Handling the Model's Requests This function runs when the model decides to use a tool. It parses the request and uses Python's built-in libraries to interact with the file system.

```

import json
import os

def handle_tool_call(tool_name, arguments):
    """Execute the requested tool and return the result as a string."""
    try:
        if tool_name == "read_file":
            with open(arguments['file_path'], 'r') as f:
                return f.read()
        elif tool_name == "write_file":
            with open(arguments['file_path'], 'w') as f:
                f.write(arguments['content'])
            return f"Successfully wrote to {arguments['file_path']}"
        # Add cases for other tools like `list_files`, `execute_python`, etc.
    except Exception as e:
        return f"Error executing tool '{tool_name}': {str(e)}"

```

3. Putting It All Together in a Conversation Loop This loop manages the back-and-forth between the user, the LLM, and your tool-execution code.

```

from llama_cpp import Llama

# Initialize your model (e.g., LFM2-1.2B-Tool)
llm = Llama(model_path="path/to/your/model.gguf", n_gpu_layers=-1, n_ctx=4096,

```

```

verbose=False)

# Start the conversation
messages = [{"role": "user", "content": "Create a file named 'groceries.txt'
with a list of 3 items."}]

# 1st call: The LLM decides if it needs to call a tool
response = llm.create_chat_completion(messages=messages, tools=tools,
tool_choice="auto")

if response['choices'][0]['message'].get('tool_calls'):
    # LLM wants to call a tool, so we execute it
    tool_call = response['choices'][0]['message']['tool_calls'][0]
    function_name = tool_call['function']['name']
    arguments = json.loads(tool_call['function']['arguments'])
    function_response = handle_tool_call(function_name, arguments)

    # We now append the model's call and our function's result to the history
    messages.append(response['choices'][0]['message'])
    messages.append({"role": "tool", "name": function_name, "content":
function_response})

# 2nd call: The LLM processes the function's result and provides a final
answer
final_response = llm.create_chat_completion(messages=messages, tools=tools)
print(final_response['choices'][0]['message']['content'])

```

The `llama-cpp-python` library provides full support for this workflow, allowing models to seamlessly use your custom tools.

⚠ Key Considerations for Real-World Use

Before you start, keep these important points in mind:

- **Autonomous Safety:** Giving an LLM access to your file system is incredibly powerful, but it also introduces significant risks. **Always run the agent in a sandboxed directory** to prevent it from accidentally or maliciously deleting important files. As a safety practice, you can implement a confirmation step for destructive operations like `write_file` or `delete_file`.
- **Model Performance:** A 1.5B model will be extremely fast (100+ tokens/second) and great for basic tasks, but for more complex, multi-step programming problems, its reasoning may be limited. In those cases, you may get better results with a larger model like the 7B-8B parameter models mentioned in the previous list.
- **Determinism:** Expect some variability. For complex tasks, the model might sometimes generate the function call in an unexpected format, requiring you to make your argument-parsing code robust. This is a known quirk of LLM-driven function calling.
- **Advanced Agents:** For more complex multi-step tasks, you might eventually want to explore frameworks like **LangChain** or pre-built file system agents that build on these same principles.

To make your agent more useful, consider expanding your toolset beyond basic file operations. For example, you can create an `execute_python` tool to run generated code in a subprocess, or use your

existing RAG pipeline to give the model awareness of your entire project's codebase.

If you decide to start building this agent, would you like a ready-to-use example script, or more detailed guidance on creating a specific tool, like the `execute_python` function?